

SAP ABAP Capstone Project

Customer Order & Revenue Analysis Report

Program: ZREVENUE_REPORT

Author	Harshil Mainra
Roll Number	2306028
Course	SAP ABAP Developer (IS30040)
Institute	KIIT University, Bhubaneswar
Batch/Year	2026
Deadline	April 21, 2026
Tech Stack	SAP ABAP, ALV Grid, Eclipse ADT, SAP BTP

1. Introduction

SAP ABAP (Advanced Business Application Programming) is the core programming language of the SAP ERP ecosystem. This capstone project demonstrates a real-world ABAP development scenario: a fully functional Customer Order and Revenue Analysis Report built using the ALV Grid framework. The project was developed using Eclipse IDE with SAP ABAP Development Tools (ADT) connected to a live SAP BTP ABAP Environment system.

1.1 Development Environment

The development was carried out on Eclipse IDE version 3.56 with SAP ADT plugin installed. The system is connected to SAP BTP ABAP Environment 2602 (TRL_EN, Client 100, System CB9980002581). This is a cloud-based SAP ABAP system providing full ABAP language support and repository management.

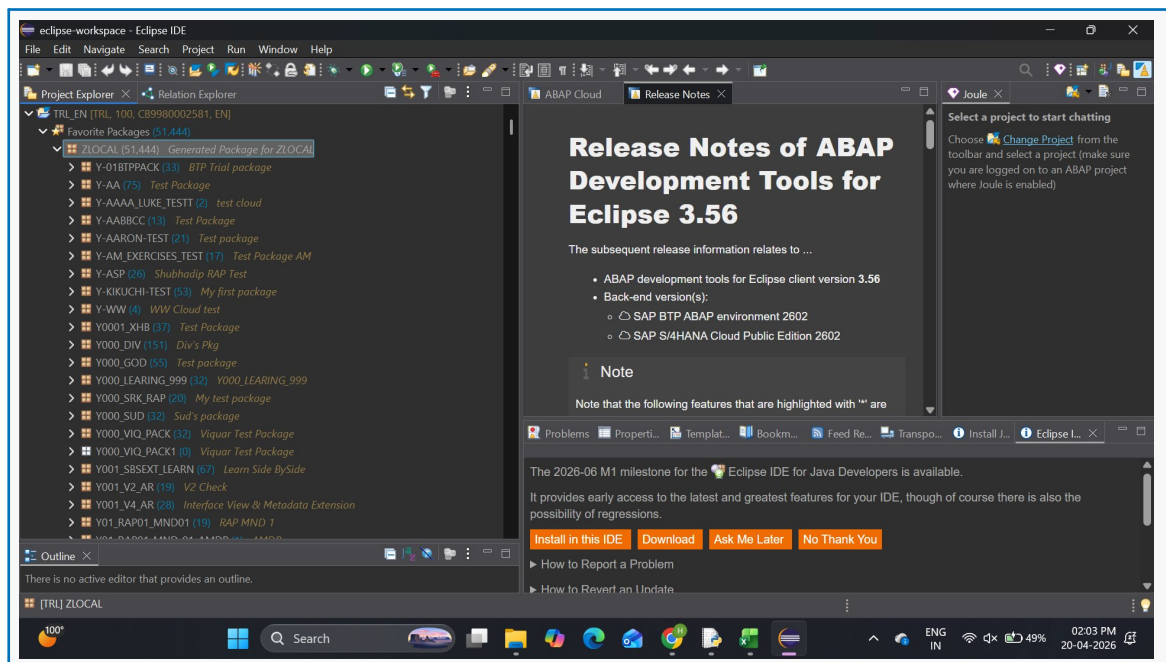


Fig 1: Eclipse IDE with SAP ADT connected to SAP BTP ABAP System (TRL_EN)

2. Problem Statement

Sales managers and finance teams in SAP-based organizations need a consolidated, filterable view of customer orders combined with revenue data. Standard SAP transactions provide basic order lists but do not combine customer master information with financial line-item data in a single interactive view.

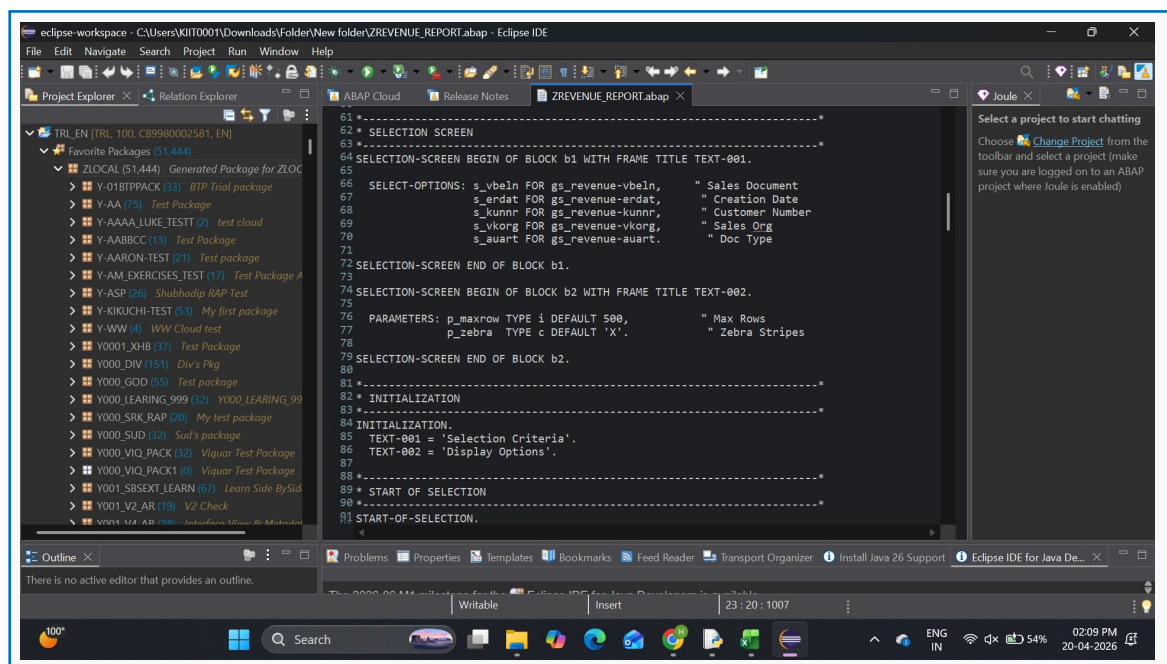
Challenge	Description
No unified view	Sales order data (VBAK/VBAP) and Customer Master (KNA1) exist in separate tables with no unified view.
No dynamic filters	Users cannot filter simultaneously by date range, customer, sales organization, and document type.
No subtotals	Revenue aggregation per customer requires manual work outside the system.
No interactivity	Standard list reports lack sorting, column resizing, and real-time refresh.

3. Solution & Features

The solution is a custom ABAP report ZREVENUE_REPORT using CL_GUI_ALV_GRID to display a joined dataset from VBAK, VBAP, and KNA1. The program is structured into modular FORM subroutines for maintainability.

3.1 Selection Screen

The report opens with a dynamic selection screen with two blocks. Block 1 contains 5 SELECT-OPTIONS for Sales Document, Creation Date, Customer Number, Sales Organization, and Document Type. Block 2 provides display options including Max Rows (default 500) and Zebra Stripe toggle.

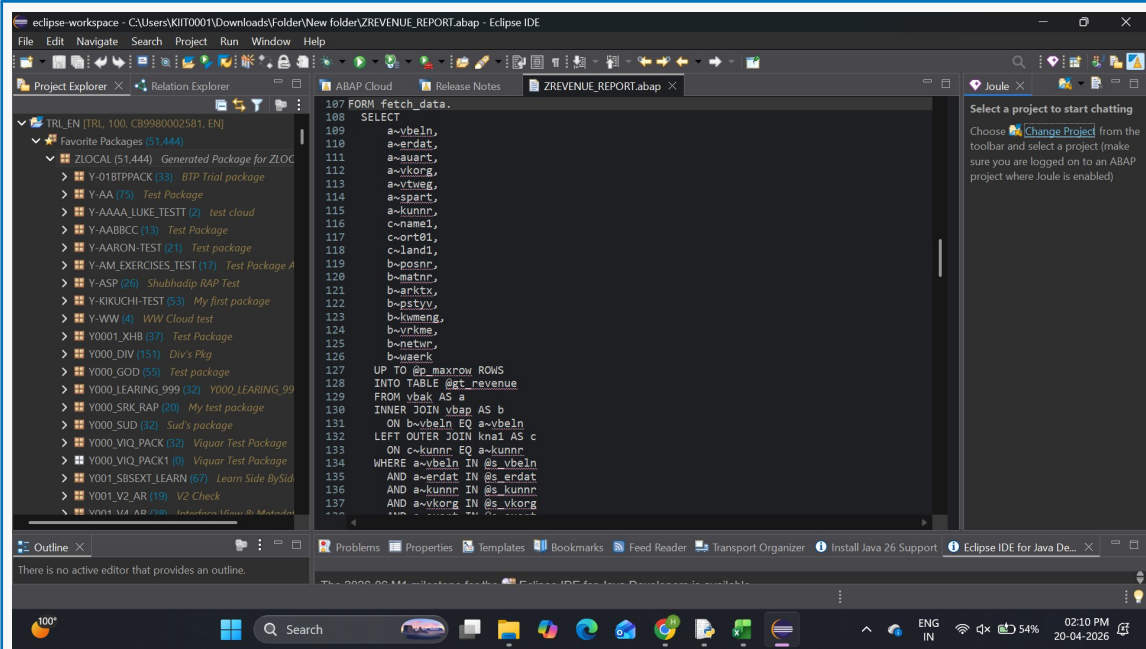


```
61 *-----*
62 * SELECTION SCREEN
63 *-----*
64 SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE TEXT-001.
65
66 SELECT-OPTIONS: s_vbeln FOR gs_revenue-vbeln,      " Sales Document
67                 s_ernat FOR gs_revenue-ernat,      " Creation Date
68                 s_kunnr FOR gs_revenue-kunnr,      " Customer Number
69                 s_vkorg FOR gs_revenue-vkorg,      " Sales Org
70                 s_auart FOR gs_revenue-auart.      " Doc Type
71
72 SELECTION-SCREEN END OF BLOCK b1.
73
74 SELECTION-SCREEN BEGIN OF BLOCK b2 WITH FRAME TITLE TEXT-002.
75
76 PARAMETERS: p_maxrow TYPE i DEFAULT 500,          " Max Rows
77              p_zebra TYPE c DEFAULT 'X'.          " Zebra Stripes
78
79 SELECTION-SCREEN END OF BLOCK b2.
80
81 *-----*
82 * INITIALIZATION
83 *-----*
84 INITIALIZATION.
85   TEXT-001 = 'Selection Criteria'.
86   TEXT-002 = 'Display Options'.
87
88 *-----*
89 * START OF SELECTION
90 *-----*
91 START-OF-SELECTION.
```

Fig 2: ABAP Selection Screen Code — 5 Dynamic Filter Parameters (Lines 62-91)

3.2 3-Table JOIN Query

Data is fetched using a single optimized SELECT with INNER JOIN on VBAK+VBAP and LEFT OUTER JOIN on KNA1. This retrieves all order line items enriched with customer name, city, and country. All WHERE conditions are driven dynamically by selection screen inputs.



```
107 FORM fetch_data.  
108   SELECT  
109     a~vbeln,  
110     a~erdat,  
111     a~vknr,  
112     a~vtwag,  
113     a~spart,  
114     a~kunnr,  
115     c~name1,  
116     c~ort01,  
117     c~land1,  
118     b~posnr,  
119     b~matnr,  
120     b~arktx,  
121     b~gstytv,  
122     b~kumeng,  
123     b~vzkm,  
124     b~netur,  
125     b~waerk  
126   UP TO @p_maxrow ROWS  
127   INTO TABLE @gt_revenue  
128   FROM vbak AS a  
129   INNER JOIN vbap AS b  
130     ON b~vbeln EQ a~vbeln  
131   LEFT OUTER JOIN kna1 AS c  
132     ON c~kunnr EQ a~kunnr  
133   WHERE a~vbeln IN @s_vbeln  
134     AND a~erdat IN @s_erdat  
135     AND a~kunnr IN @s_kunnr  
136     AND a~vknr IN @s_vknr  
137
```

Fig 3: ABAP SELECT with 3-Table JOIN — VBAK + VBAP + KNA1 (Lines 107-137)

4. Program Structure & Code

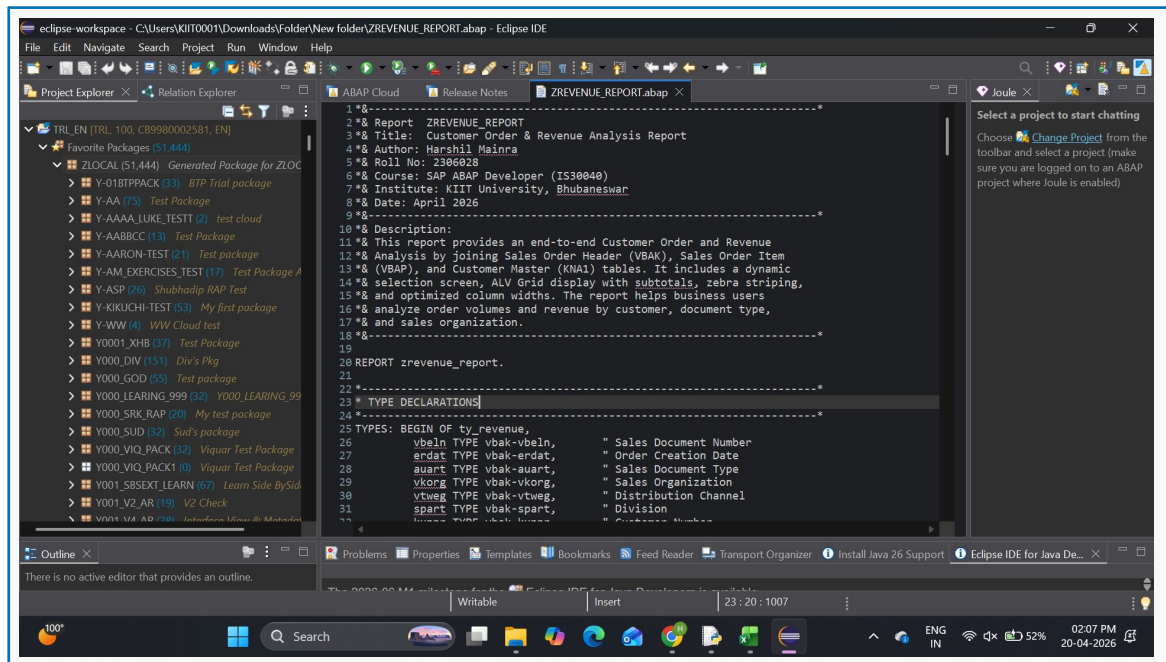


Fig 4: ZREVENUE_REPORT — Program Header, Author Details & TYPE Declarations (Lines 1-32)

4.1 ALV Grid Display

The ALV Grid is created using `CL_GUI_CUSTOM_CONTAINER` and `CL_GUI_ALV_GRID`. A custom field catalog (18 fields) is built programmatically with headers, widths and justification. Subtotals are enabled on `NETWR` (revenue) and `KWMENG` (quantity), grouped by customer number. Sort criteria are applied automatically on customer and sales document.

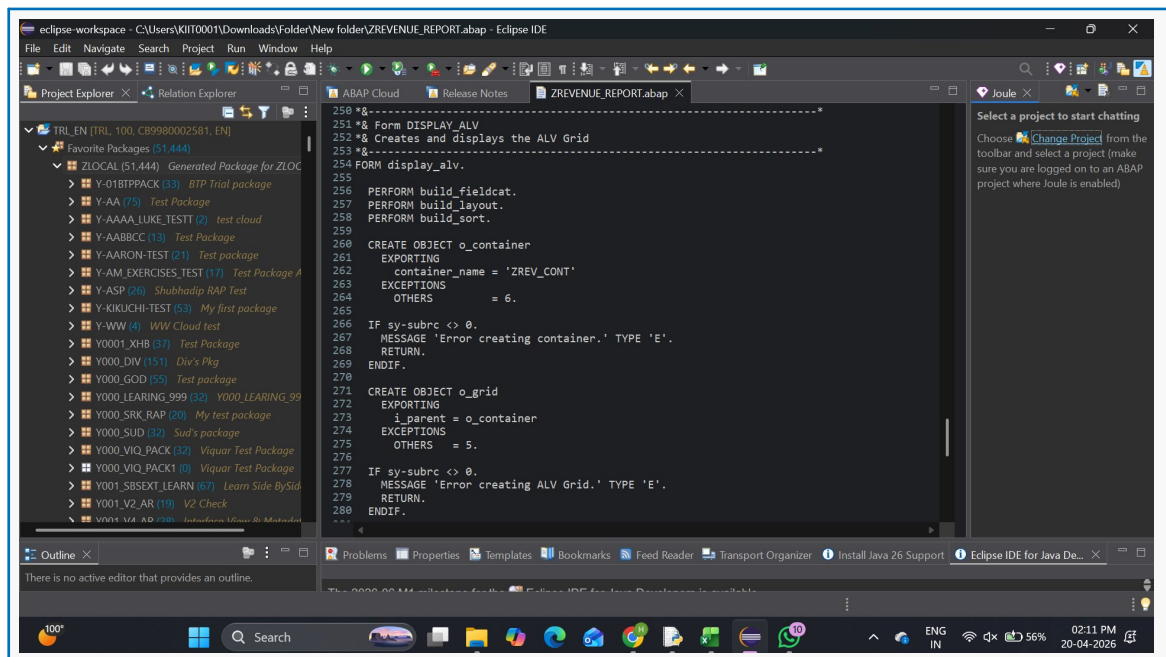


Fig 5: DISPLAY_ALV Form — `CL_GUI_CUSTOM_CONTAINER` and `CL_GUI_ALV_GRID` Creation (Lines 250-280)

FORM / Section	Purpose
TYPE/DATA	Structure ty_revenue (18 fields), internal table, ALV objects
SELECTION-SCREEN	5 SELECT-OPTIONS + Max Rows + Zebra Stripe parameters
FETCH_DATA	3-table SELECT with dynamic WHERE from selection screen
CHECK_DATA	Record count and cumulative revenue calculation
BUILD_FIELDCAT	18-field catalog using DEFINE macro, subtotals on NETWR/KWMENG
BUILD_LAYOUT	Zebra, column optimization, selection mode, grid title
BUILD_SORT	KUNNR (subtotal) + VBELN sort criteria
DISPLAY_ALV	Container + Grid creation, set_table_for_first_display
STATUS_0100	Sets PF-Status, Titlebar, calls DISPLAY_ALV
USER_COMMAND	Handles BACK/EXIT/CANC and REFR (refresh grid)

5. Technology Stack

Language	SAP ABAP (Open SQL, ABAP Objects)
IDE	Eclipse IDE 3.56 with SAP ADT Plugin
SAP System	SAP BTP ABAP Environment 2602 (Cloud)
System ID	TRL, Client 100, CB9980002581
UI Framework	CL_GUI_ALV_GRID (ALV Grid Control)
Screen	Custom Dynpro Screen 100 with Container
DB Tables	VBAK, VBAP, KNA1 (SAP SD Standard Tables)
SAP Module	SD — Sales and Distribution

5.1 SAP ADT Development Environment

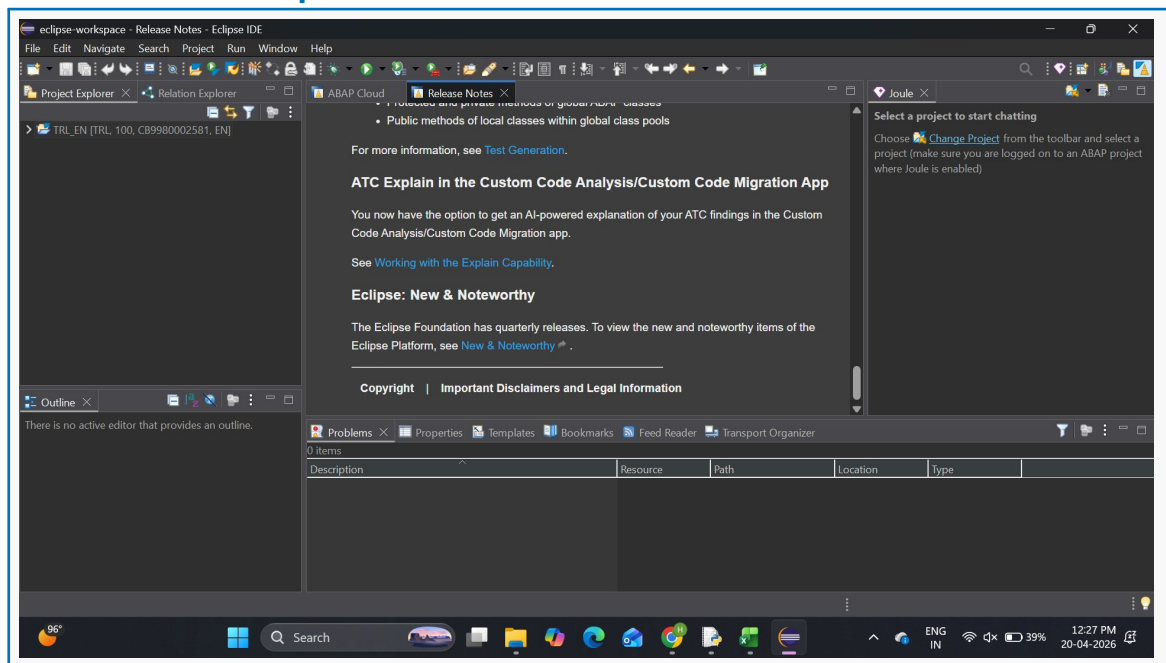


Fig 6: Eclipse ADT — ABAP Development Tools Release Notes (Version 3.56, SAP BTP 2602)

T-Code	Purpose
SE38	ABAP Editor — write and activate reports
SE11	Data Dictionary — inspect table structures
SE51	Screen Painter — design custom dynpro screens
SE41	Menu Painter — create GUI Status with function keys
SE80	Object Navigator — manage all ABAP objects
VA01	Create Sales Order — generate test data
VA03	Display Sales Order — verify order data

6. Unique Points

#	Feature	Description
1	3-Table JOIN	Joins VBAK+VBAP+KNA1 — includes Customer Name, City, Country. Most basic report
2	Dynamic Selection Screen	5 independent SELECT-OPTIONS allow flexible filtering by any combination of criteria.
3	Programmatic Field Catalog	16 fields built via DEFINE macro — full control over headers, widths, and justification per
4	Customer Subtotals	do_sum on NETWR and KWMENG with sort subtotals on KUNNR produces automatic p
5	Modular Code Structure	7 FORM subroutines each with single responsibility — fetch, validate, fieldcat, layout, so
6	Refresh Without Re-run	F5 REFR command calls refresh_table_display after re-fetch — no need to re-execute th

7. Future Improvements

Enhancement	Description
Summary Tab	Second ALV showing grouped revenue totals per customer alongside detail view.
Delivery Status	Include VBUP table to show per-item delivery and billing status.
Excel Export	Enable ALV built-in Excel export via toolbar configuration.
Top-of-Page Header	Add company logo, run timestamp, and record/revenue summary above the grid.
Drill-Down	Double-click VBELN hotspot to navigate directly to VA03 for that order.
Variant Management	Save/restore ALV column layouts using SAP variant management.

8. Conclusion

This capstone project successfully implements a real-world SAP ABAP development scenario — a Customer Order and Revenue Analysis Report (ZREVENUE_REPORT). The project was developed using Eclipse IDE with SAP ADT connected to a live SAP BTP ABAP Environment, demonstrating practical cloud-based SAP development.

The report covers the complete ABAP development lifecycle: data modelling (3-table joins and structure definitions), UI design (selection screen and ALV Grid), screen programming (dynpro and GUI status), and modular code organization across 7 FORM subroutines. The use of programmatic field catalog, customer-level subtotals, dynamic filters, and a refresh mechanism demonstrates capabilities well beyond basic report programming.

Skills gained through this project — SAP ABAP Open SQL, ALV Grid Control, Dynpro programming, table joins, and BTP cloud development — are directly applicable in enterprise SAP developer roles.

9. References

- SAP Help Portal — ABAP Keyword Documentation: <https://help.sap.com/doc/abapdocu>
- SAP ADT (ABAP Development Tools) for Eclipse: <https://tools.hana.ondemand.com>
- SAP BTP ABAP Environment Documentation: <https://help.sap.com/docs/btp>
- KIIT Kareer School — SAP ABAP Developer Course Material (IS30040), 2026
- SAP Standard Table Reference (VBAK, VBAP, KNA1) — SE11 Data Dictionary